

Práctica 1.2

Ejemplo con MPI

ÍNDICE

1	OBJETIVOS Y PUNTUACIÓN.....	2
2	MODELO DE PARALELIZACIÓN	2
3	DISEÑO EN MPI.....	4
3.1	Esquema de procesos.....	4
3.2	Tipo de mensajes.....	7
3.3	Descomposición modular	7
4	IMPLEMENTACIÓN.....	8
5	PRUEBAS.....	9

1 OBJETIVOS Y PUNTUACIÓN

Los principales objetivos de esta práctica son:

- Servir de modelo para el desarrollo de la práctica 1.3 que deberá realizar cada equipo.
- Implementar un algoritmo paralelo de ordenación de una secuencia de números enteros.
- Familiarizarse con el entorno MPI, utilizado en el desarrollo de la práctica.
- Probar ejecuciones en más de un procesador físico y observar cómo de buena es la paralelización en relación con la solución secuencial de este mismo problema.
- Afianzar los conocimientos sobre aceleración y eficiencia.

Esta práctica vale 0.25 puntos de los 5.5 puntos de prácticas de la asignatura.

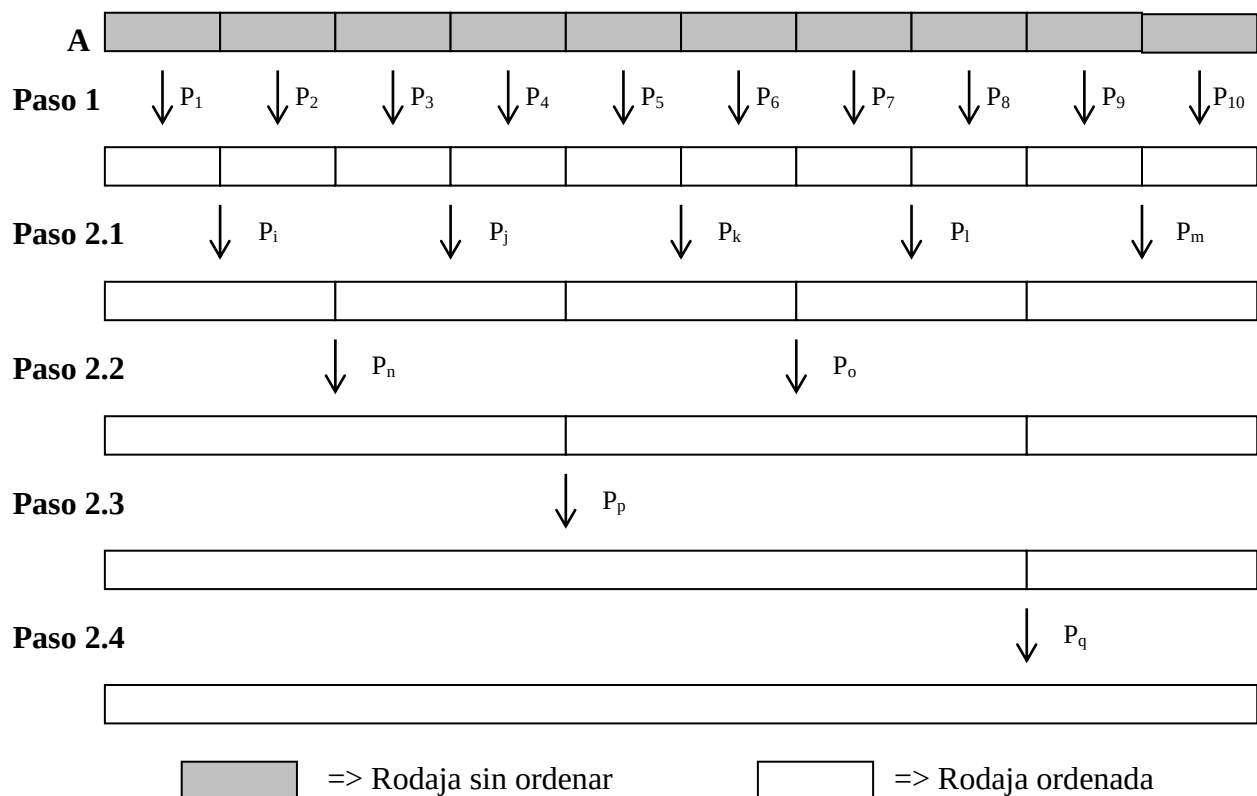
2 MODELO DE PARALELIZACIÓN

Partiremos del supuesto de que se dispone de N procesadores y hay que ordenar un vector A de dimensión " r ".

La idea intuitiva de paralelización es muy sencilla y consta de dos pasos:

1. Se divide A en N trozos y cada procesador ordena una de las partes utilizando el algoritmo de la burbuja. Se generan N rodajas ordenadas.
2. Utilizando un algoritmo de mezcla secuencial, se mezclan dos a dos las rodajas ordenadas en el paso 1, hasta conseguir una única secuencia ordenada.

Gráficamente, para $N=10$, sería:

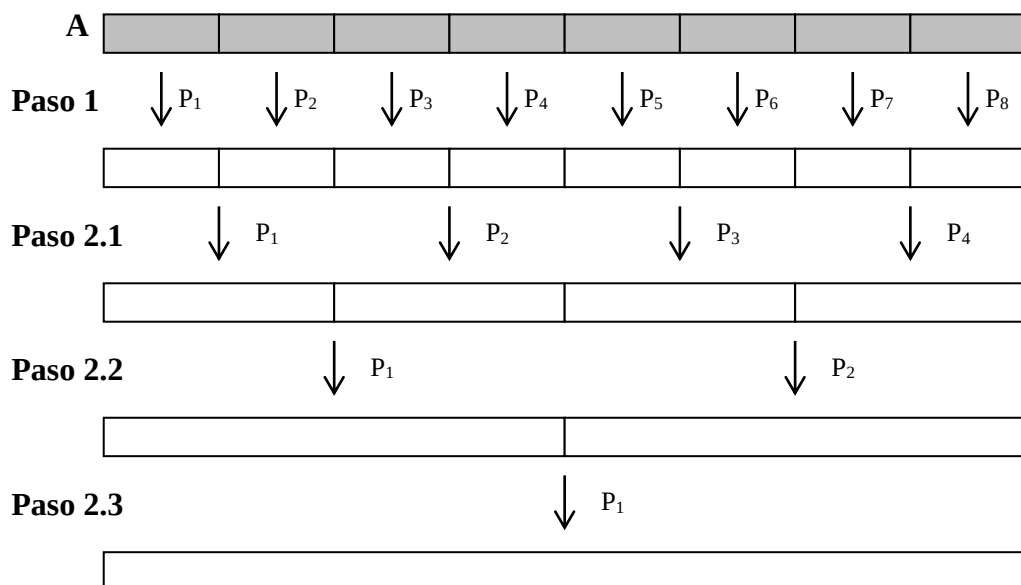


Este modelo nos plantea, al menos, los dos problemas siguientes:

- En el segundo paso (que a su vez se descompone en subpasos), hay subpasos que parten de un número par de rodajas (como 2.1 y 2.4) y subpasos que parten de un número impar de rodajas, haciendo que **la solución no pueda ser regular**.
- El número de procesadores que tiene trabajo en cada subpaso es distinto y decreciente. Hay que tomar una decisión acerca de **qué procesos hacen qué trabajo**.

Para resolver el primer problema, se decide restringir la solución de tal forma que el número de procesadores disponibles sea una potencia de 2. De esta forma, el número de subpasos necesarios para hacer todas las mezclas necesarias será de $\log_2 N$, todas ellas regulares ya que siempre habrá un número par de rodajas a mezclar.

Para resolver el segundo problema, hay varias alternativas. Una de ellas se pone gráficamente a continuación, supuesto que el número de procesadores “N” es igual a 8:



En esta solución, se ha decidido que cuando hay trabajo para “P” procesadores, sean los P procesadores cuyo identificador es menor los que se encarguen de hacerlo. Pero igualmente podría haberse decidido que fuesen los de mayor peso o cualquier otra combinación. En cualquier caso, interesa que esta asignación sea regular.

Este modelo de solución que vamos a desarrollar es **claramente ineficiente** ya que no distribuye la carga de trabajo del problema de forma homogénea entre los procesadores disponibles. En el primer paso tenemos trabajo para N procesadores, en el siguiente para N/2 procesadores, en el siguiente para N/4 procesadores y así sucesivamente hasta que sólo queda trabajo para el procesador P_1 . Se deja al alumno que piense en soluciones mejores.

Esta es una solución todavía sin pensar en el modelo de procesamiento que hay por debajo. Si acaso, está más cercano al modelo de multiprocesadores (memoria común) que de multicomputadores.

3 DISEÑO EN MPI

Este tipo de problema, a no ser que se trabaje con vectores muy grandes, no es excesivamente intensivo en cálculo, ya que las operaciones elementales sobre cada dato o pareja de datos es una comparación y una copia sobre información de tipo entero.

Dado que se desea comprobar si la paralelización hace o no una buena distribución de la carga de trabajo en base a mediciones del tiempo de ejecución, se tomó la decisión de utilizar el algoritmo de la burbuja en vez del *quicksort* por ser el primero un algoritmo de complejidad cuadrática y ofrecer tiempos de varios segundos para ordenar un vector de 100.000 enteros, mientras que con el *quicksort* los tiempos son de unos pocos milisegundos en un procesador moderno.

Las cuestiones de diseño que se van a tratar a continuación son:

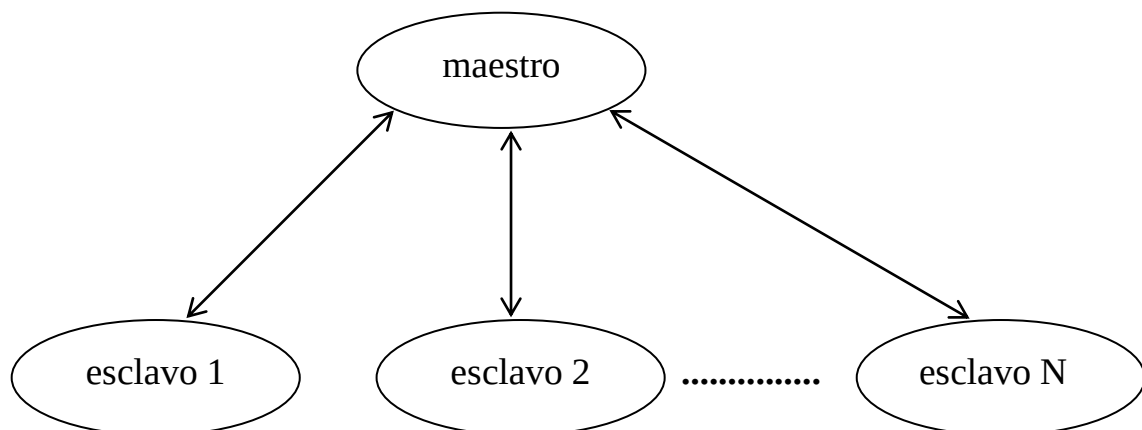
- El esquema de procesos.
- El tipo de mensajes intercambiados entre los procesos.
- La descomposición en módulos.

3.1 Esquema de procesos

La aplicación se descompone en dos tipos de procesos: **maestro** y **esclavos**.

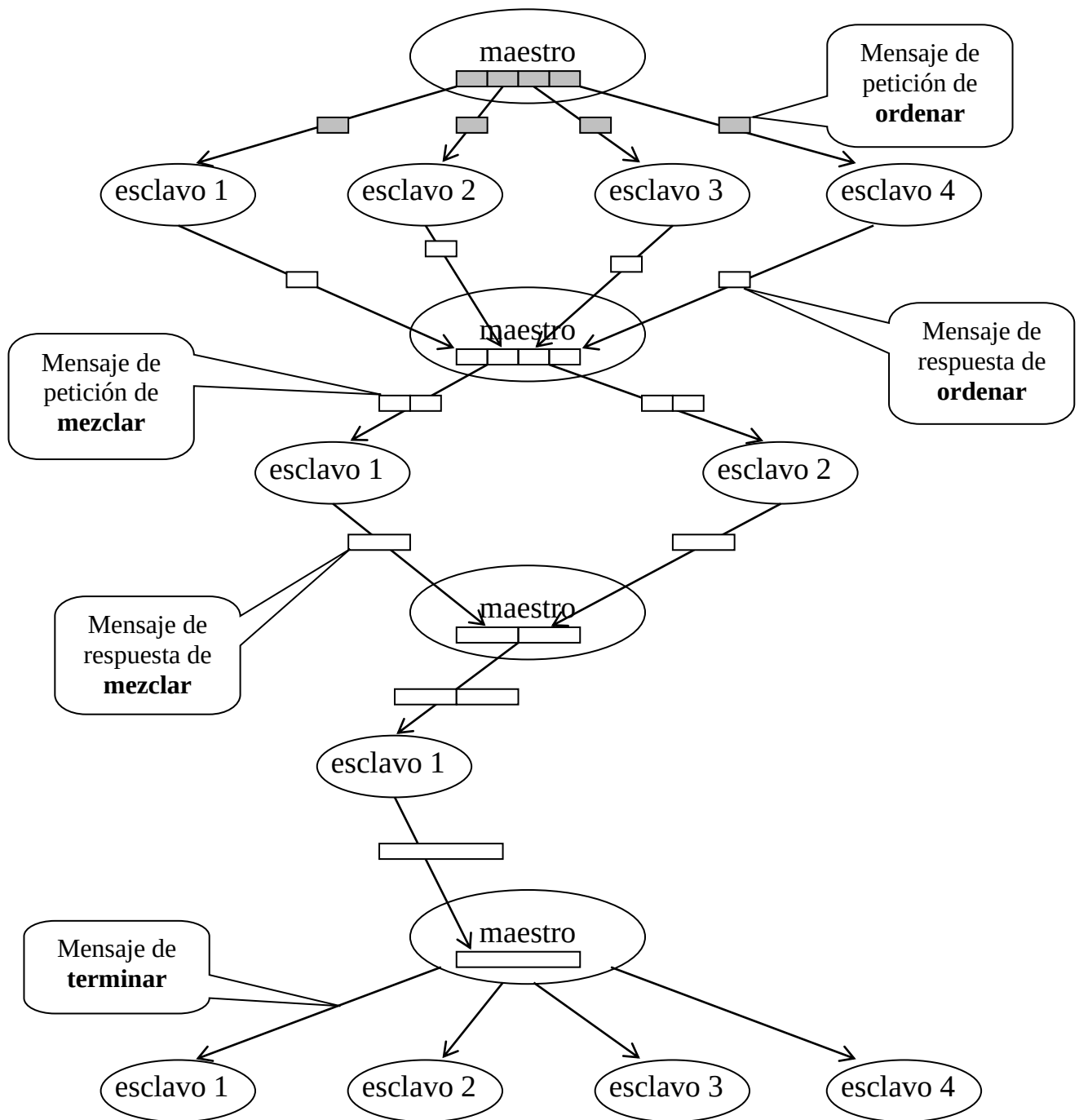
Habrà un único proceso maestro que se encargará de partir el trabajo, enviárselo a los esclavos y recoger el trabajo realizado por los mismos. En definitiva, un modelo típico maestro-esclavos.

Gráficamente queda lo siguiente:



Una primera aproximación sería aquella en la que el maestro tuviese toda la inteligencia de distribución del trabajo y, por lo tanto, enviase a los esclavos dos tipos de peticiones: **ordenar** una secuencia y **fusionar** dos subsecuencias. En ambos casos, el maestro recogería el resultado (una secuencia ordenada), para a su vez seguir distribuyendo más trabajo hasta conseguir tener el vector **A** totalmente ordenado, momento en el que el maestro enviaría una petición de **terminar** a los esclavos.

Este proceso se representa gráficamente a continuación, supuesto $N=4$.

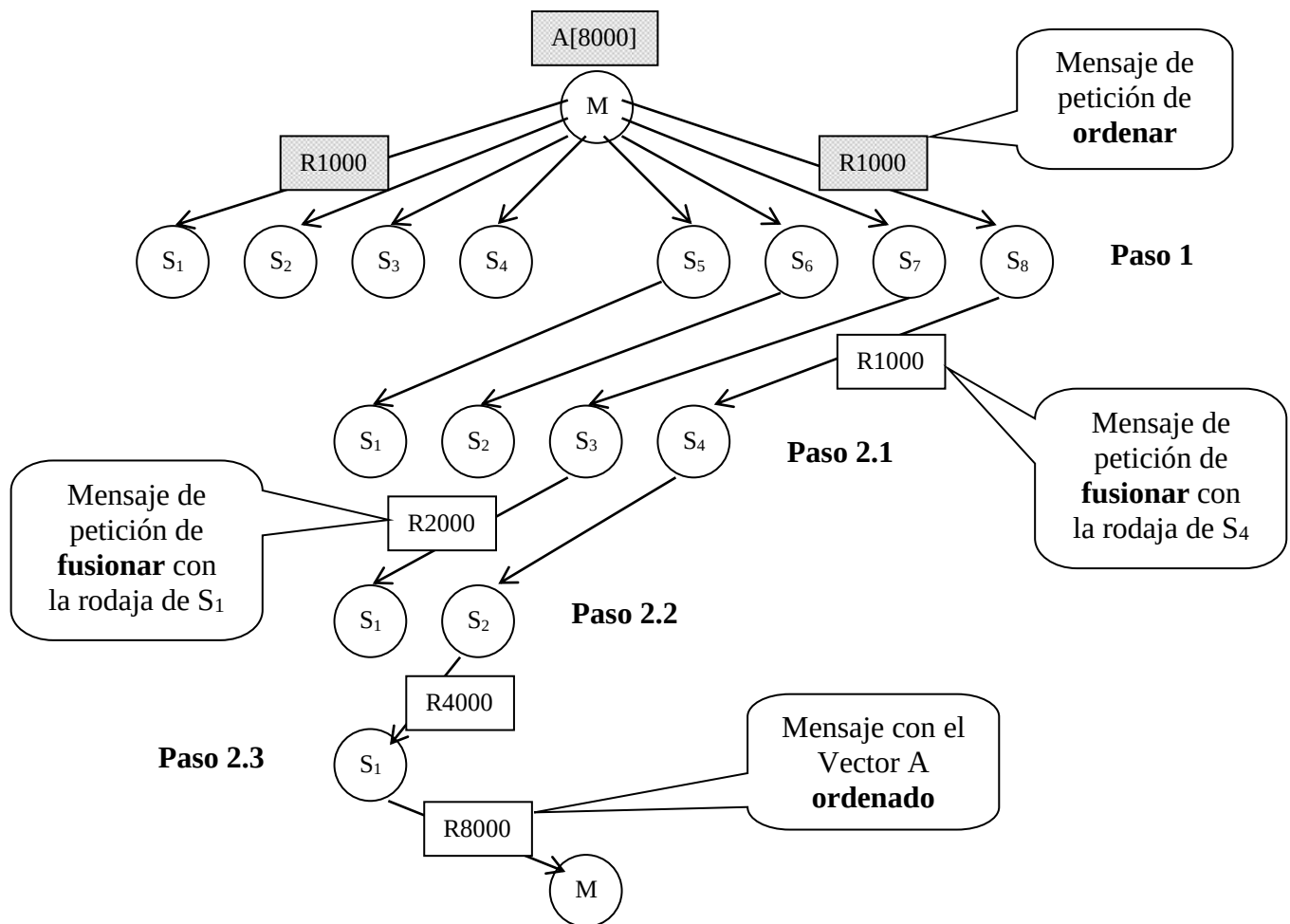


Esta solución tiene el inconveniente de necesitar enviar muchos mensajes ya que van mensajes de petición hacia los esclavos, que responden con mensajes de respuestas, que agrupados vuelven a enviarse a esclavos y así sucesivamente.

La solución que finalmente se va a implementar va a consistir en dotar de inteligencia a los esclavos de forma que ellos mismos sean los que se distribuyan el trabajo de las mezclas sin necesitar de la intervención del maestro.

Para ello, cada esclavo sabrá qué esclavo es (1, 2, 3, etc.). Inicialmente recibirán todos ellos una petición del maestro de ordenar una rodaja. Una vez ordenada, se las pasarán entre sí para conseguir la ordenación total.

Uno de los mecanismos posibles de irse pasando las rodajas mezcladas es el que se pone a continuación, donde **M** indica proceso **Maestro**, **Pi** **Proceso iésimo** y **R1000** una rodaja de 1000 elementos (se parte de un vector A de 8000 elementos):



Con esta solución, en el Paso 2.1, los procesos S₅, S₆, S₇ y S₈ deben pasar la rodaja que ordenaron en el Paso 1 a los procesos S₁, S₂, S₃ y S₄ respectivamente y terminar (ya no tienen más trabajo que hacer). Por su parte, los procesos S₁, S₂, S₃ y S₄ deberán mezclar la rodaja que ordenaron en el Paso 1 con la que les pasa el proceso S₅, S₆, S₇ o S₈ respectivamente.

En el Paso 2.2 pasarán rodaja S₃ y S₄ a S₁ y S₂ respectivamente y terminarán. En el Paso 2.3 será S₂ el que le pase la rodaja a S₁ y terminará. Al final del Paso 2.3, S₁ tendrá el vector totalmente ordenado y se lo pasará al maestro, tras lo que también terminará.

Cuando el maestro recibe el vector ordenado, simplemente lo mostrará y también podrá terminar sin tener que preocuparse por los procesos esclavos ya que todos ellos han ido terminando de una forma ordenada.

Finalmente, comparando esta solución con la primera propuesta, vemos que la primera supondría (para este caso de N=8 y r=8000) el envío de 30 mensajes con un total de 64 000 números, mientras que la solución adoptada supone el envío de 16 mensajes con un total de 28 000 números.

3.2 Tipo de mensajes

Lo cierto es que hay tres tipos de mensajes:

- Mensajes de petición de **ordenar** una rodaja.
- Mensajes de petición de **fusionar** una rodaja con la rodaja del destinatario del mensaje.
- Mensaje de **vector ordenado**.

Sin embargo, todos ellos tienen la misma estructura (un vector de números enteros). Además, cada proceso sabe en todo momento qué tipo de mensaje debe enviar o recibir.

Por ejemplo, el maestro enviará al principio N mensajes de ordenar, uno a cada proceso esclavo que haya creado, pasando a continuación a esperar el mensaje con el vector ordenado que le debe enviar el primer servidor.

Todos los esclavos esperarán inicialmente recibir el mensaje de petición de ordenar una rodaja que les envía el maestro. Posteriormente, si se sabe determinar en cada subpaso qué proceso actúa como emisor de rodaja a fusionar (y por lo tanto debe terminar tras el envío) y qué proceso actúa como receptor de dicha rodaja (que debe fusionar con la que él tenga), está claro que se estarán enviando y recibiendo mensajes de petición de fusionar de forma ordenada.

Finalmente, el primer servidor, al saber que ya ha conseguido fusionar las dos últimas rodajas, enviará el mensaje con el vector ordenado.

En la parte de implementación se tratará el detalle de cómo los esclavos saben qué esclavo concreto es cada uno de ellos, así como el número de subpasos para resolver el problema y si tienen que enviar o recibir mensaje de fusión en cada subpaso.

La conclusión es que es suficiente con definir un único tipo de mensaje cuya estructura sería:

```
int longR;    {longitud de la rodaja (a ordenar) | (a fusionar) | (ordenada) }  
int R[longR]; { rodaja a ordenar }
```

3.3 Descomposición modular

La funcionalidad del proceso maestro se realiza en un único módulo (fichero):

- **Maestro**, que es el principal y se encarga de generar el vector A con números aleatorios, crear los esclavos, enviarles la rodaja a ordenar y esperar por el resultado.

La funcionalidad de los procesos esclavos se realiza en dos módulos:

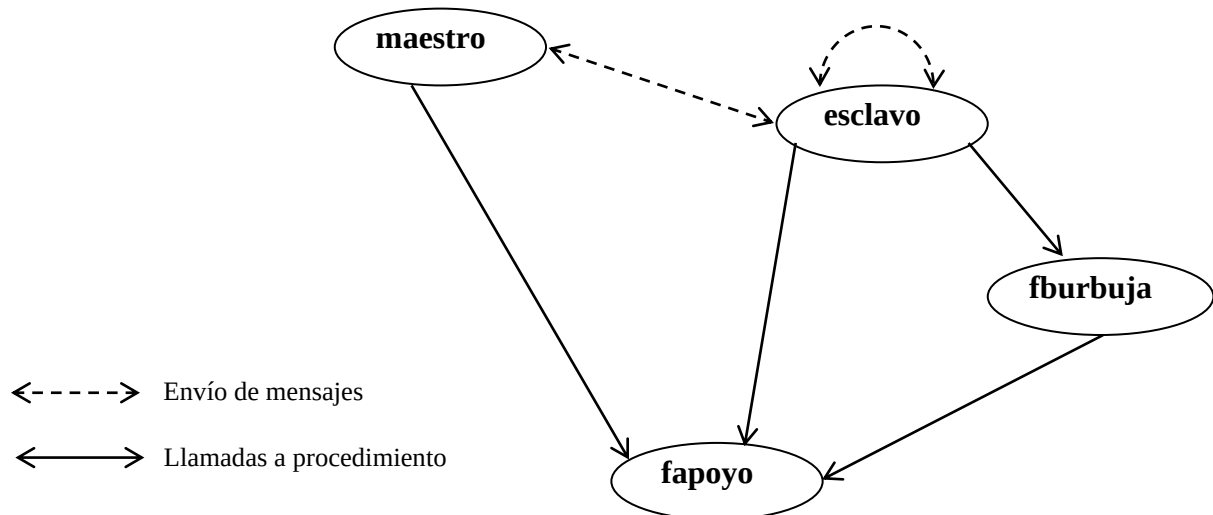
- **Esclavo**, que es el principal y se encarga de recibir los mensajes de petición de trabajos y hacer que los mismos se realicen invocando a las funciones exportadas por el otro módulo.
- **Fburbuja**, que implementa el algoritmo de la burbuja en versión secuencial.

Tanto el maestro como el esclavo hacen uso del módulo:

- **Fapoyo**, que exporta las funciones:

- **init_aleatorios(rango)**: que inicia una secuencia de aleatorios del rango indicado con semilla dependiente del instante de la llamada.
- **sec_aleatoria(S, longS)**: que rellena **S** con una secuencia aleatoria de **longS** números enteros dentro del rango establecido por **init_aleatorios**.
- **visualiza_rodaja(S, longS)**: que visualiza los números de la rodaja **S**.

El grafo de dependencias entre módulos es el siguiente:



4 IMPLEMENTACIÓN

Todos los ficheros con fuentes en C, así como el **Makefile**, están disponibles en el directorio p1.2 del servidor.

Para la implementación en C usando además la biblioteca de MPI, se ha generado un fichero **Makefile** que recoge el grafo de dependencias entre módulos, así como la de un módulo más, utilizado en la fase de pruebas, que permite lanzar la ejecución puramente secuencial del algoritmo de ordenación por el método de la burbuja.

En el fichero **practica.h** se incluyen definiciones generales de la práctica para identificar el número máximo de procesos esclavos que se lanzarán, así como el tamaño del vector a ordenar.

De los módulos que no son principales, se ha generado el correspondiente fichero de cabecera, por ejemplo, **fapoyo.h**, conteniendo la especificación de las funciones exportadas.

Los programas en C son autoexplicativos, tan sólo resaltar cómo se consigue que los esclavos sepan qué tienen que hacer en cada paso: todos los esclavos conocen el tamaño del problema, especialmente el número de esclavos (**N**) que coincidirá con el número inicial de rodajas a ordenar y fusionar en un primer paso. El paso 2 no dejará de ser un bucle en el que primero hay **N** rodajas por mezclar, luego **N/2**, después **N/4** y así sucesivamente hasta que sólo quede una rodaja que será la solución.

En cada uno de esos subpasos, habrá unos procesos esclavos que actúen como **mezcladores** y otros tantos que actúen como **suministradores** de rodaja para mezclar. Para el caso de que hubiese 16 procesos esclavos, el papel de cada uno de ellos en cada subpaso sería el siguiente:

Rodajas a fusionar	P. Mezcladores	P. Suministradores	$P_j \leftarrow P_i$	$P_i \rightarrow P_j$
16	1..8	9..16	$j \leftarrow j + 8$	$i \rightarrow i - 8$
8	1..4	5..7	$j \leftarrow j + 4$	$i \rightarrow i - 4$
4	1..2	3..4	$j \leftarrow j + 2$	$i \rightarrow i - 2$
2	1	2	$j \leftarrow j + 1$	$i \rightarrow i - 1$

Es decir, si quedan R rodajas:

- Los procesos cuyo identificador sea menor que R/2 serán mezcladores y el resto suministradores.
- Los mezcladores recibirán una rodaja del suministrador cuyo identificador sea R/2 más alto que el suyo, pasando a mezclar la rodaja recibida con la suya.
- Los suministradores enviarán su rodaja al mezclador cuyo identificador sea R/2 más bajo que el suyo, tras lo que terminarán.

Cuando sólo quede una rodaja, el único proceso servidor existente será el uno que tendrá todo el vector ordenado y se lo enviará al maestro.

5 PRUEBAS

Todas las pruebas de medición de tiempos se realizarán con un vector de 128 000 enteros.

Para comprobar que la solución es correcta, se ha probado con un vector de 100 números y se comprobó visualmente que ordenaba correctamente. Además, con las pruebas de 128 000 enteros se comprobó visualmente que la secuencia sale ordenada y se comprobó al azar algunos números observando que aparecían el mismo número de veces en la secuencia ordenada que en la secuencia desordenada. Con un poco más de esfuerzo podría haberse hecho una comprobación automática con filtros Unix (*cut*, *sort*, etc...). El alumno puede suponer que estas pruebas ya están hechas.

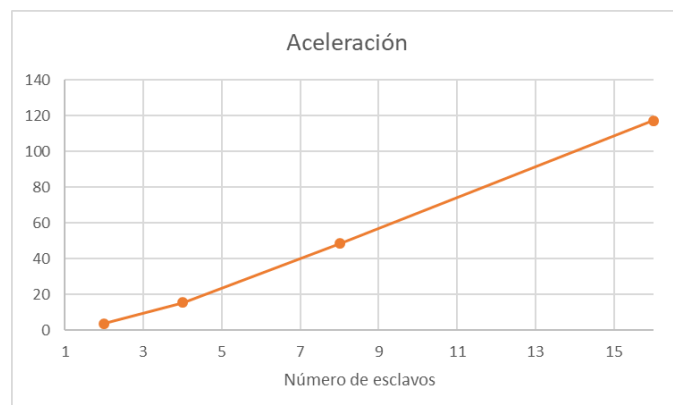
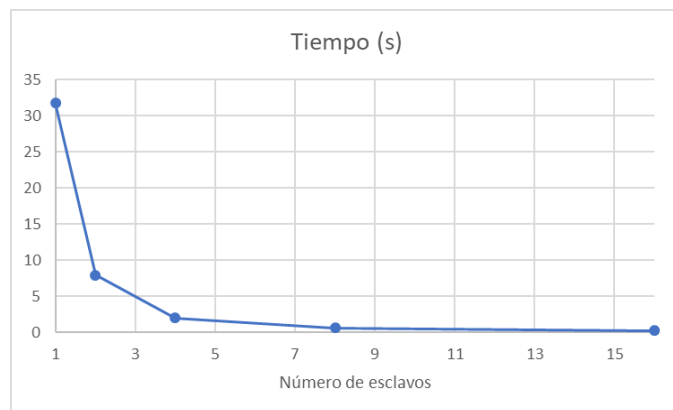
Todos los ficheros necesarios para ejecutar las pruebas están disponibles en el servidor en el directorio **/usuarios/alumnosArqava/arqava00/p1.2**

Para afianzar los conceptos de MPI y los conceptos de aceleración y eficiencia explicados en clase de teoría, cada pareja de alumnos debe calcular estos valores con 2, 4, 8 y 16 procesadores. Para ello, debe empezar ejecutando la versión secuencial (programa **burbuja**), que sólo usa un procesador. Como puede haber bastante variación en los tiempos, ejecutar tres veces y tomar la media de los tres.

A continuación, usando MPI para ejecutar los procesos **maestro** y **esclavo**, ejecutar con 2, 4, 8 y 16 procesadores para los esclavos, usando hasta 4 computadores del laboratorio. Esto quiere decir que, por ejemplo, cuando se haga una prueba con 8 procesadores, se deben lanzar 2 procesos esclavo en cada computador y el maestro estará en el mismo ordenador que dos de los esclavos, ya que el total de procesos será $1+4 \times 2=9$. Apuntar los tiempos en una tabla como la siguiente (los tiempos son ejemplos):

#esclavos	Tiempo (s)	Aceleración	Eficiencia
1	31.783		
2	7.989	3.98	1.99
4	2.028	15.67	3.92
8	0.654	48.60	6.07
16	0.271	117.28	7.33

Se recomienda apuntar los tiempos en una hoja de cálculo, ya que se deberán hacer tres gráficos como los siguientes:



A entregar – 1. Tabla de mediciones (5 puntos)

Realiza una **captura de pantalla** donde se vea la ejecución de la medición con 16 esclavos, incluyendo la orden usada y el resultado. Recuerda aumentar el tamaño de la terminal si se ve muy pequeña en la captura.

Incluir la **tabla** indicada con mediciones.

Incluir también las **órdenes** usadas para cada medición.

A entregar – 2. Gráficas (2.5 puntos)

Incluir gráficas como las indicadas anteriormente.

A entregar – 3. Análisis (2.5 puntos)

Analizar los resultados indicando si la aceleración es mejor, peor o igual a la ideal y por qué.